

Project Planning Phase

Planning Logic

Agile Project Planning & Sprint Logic

1. Agile Framework Definitions

To ensure a structured implementation and predictable delivery timeline, the development of the platform adheres to standard Scrum and Agile methodologies:

Sprint: A fixed period or duration (typically 2 weeks) in which the cross-functional engineering team works to complete a committed set of automation tasks.

Epic: A large, high-level business capability or project phase that is too massive to complete in a single sprint. Epics are broken down into smaller, actionable User Stories.

User Story (Story): A small, discrete unit of development work written from an end-user or system perspective that can be completed within a single sprint block.

Story Point: A relative number representing the total effort, complexity, and risk a user story takes to complete, estimated using a modified Fibonacci sequence (1, 2, 3, 5, 8).

1 Point: Very Easy Task (Basic UI text adjustment, static property configuration).

2 Points: Normal Task (Standard Client Scripting, simple form variable rendering).

3 Points: Moderate Task (Multi-condition business rule, relational lookup script).

5 Points: Difficult Task (Complex asynchronous workflow loops, external REST API integration spoke).

○

2. Sprint Roadmap & Backlog Distribution

Sprint 1: Front-End Ingestion & Core Logic Pipeline

Focus: Establishing the presentation layer, form sanitization barriers, and initial server-side processing routings.

Epic 1: Presentation & Ingestion Tier

Story USN1: Develop Service Portal Catalog Item Form Layout (HTML5/CSS3) --> **2 Story Points** (*Normal*)

Story USN2: Configure basic system profile lookup scripts for requester session auto-population --> **1 Story Point** (*Very Easy*)

Epic 2: Ingestion Logic & Parameter Sanitization

Story USN3: Implement Catalog Client Scripts for real-time string formatting verification ---> **3 Story Points** (*Moderate*)

Story USN4: Engineer UI Policies for contextual field visibility (VLAN tags, IP Subnets)---> **3 Story Points** (*Moderate*)

Story USN5: Build synchronous business rules to catch data anomalies before database submission --> **3 Story Points** (*Moderate*)

Total Story Points committed in Sprint 1: $2 + 1 + 3 + 3 + 3 = 12$ **Story Points**

Sprint 2: Workflow Automation, Ledger Capture & Integrations

Focus: Activating asynchronous manager approval cards, custom database commits, and external API gateway hooks.

Epic 3: Asynchronous Automation Engine

Story USN6: Build multi-stage routing loop inside Flow Designer for manager assignments --> **2 Story Points** (*Normal*)

Story USN7: Generate dynamic approval card visualization components for the workspace --> **2 Story Points** (*Normal*)

Story USN8: Develop backend Script Includes for automated engineering task decomposition --> **2 Story Points** (*Normal*)

Story USN9: Optimize table indexing structures and client-side variable caching keys --> **4 Story Points** (*High Moderate*)

Epic 4: Data Persistence Layer

Story USN10: Construct custom database schema ledger registry tables (u_network_database) --> **5 Story Points** (*Difficult*)

Epic 5: External API Interfaces

Story USN11: Configure IntegrationHub REST/SOAP Spoke bindings for third-party orchestration --> **5 Story Points** (*Difficult*)

Total Story Points committed in Sprint 2: $2 + 2 + 2 + 4 + 5 + 5 = 20$ Story Points

3. Team Velocity & Capacity Calculations

Team velocity measures how much work an engineering squad can reliably complete within a single sprint block. This metric is critical for forecasting future deployment milestones.

❑ Total Velocity Formulation

Velocity = Total Story Points Completed/Number of Sprints

Total Project Backlog Effort: 12 (Sprint 1) + 20 (Sprint 2) = 32 Total Story Points

Observed Sprint Lifecycles: 2 Sprints

Velocity Computation:

Velocity = $32 / 2 = 16$ Story Points per Sprint

Project Planning Baseline: The engineering team's established velocity is **16 Story Points per Sprint**. Future platform feature iterations, dashboard additions, or secondary API spoke expansion phases will be mapped against this capacity index to guarantee predictable launch schedules.